

1. What is Logging in ASP.NET Core?

Logging is the process of recording application events (information, warnings, errors, and critical failures) during runtime.

In ASP.NET Core, logging helps you:

- Diagnose errors and exceptions
- Track application behavior
- Monitor performance and failures
- Audit important business operations

ASP.NET Core provides a **built-in, extensible logging framework** via `Microsoft.Extensions.Logging`.

2. Built-in Logging Architecture (High Level)

ASP.NET Core logging consists of:

- **ILogger** – Used to write log messages
- **ILoggerFactory** – Creates loggers
- **Log Providers** – Decide where logs go (Console, Debug, File, Seq, Application Insights, etc.)

Default providers include:

- Console
 - Debug
 - EventSource
-

3. Log Levels (Most Important Concept)

ASP.NET Core supports the following log levels (in increasing severity):

Level	Usage
Trace	Very detailed, rarely used

Level	Usage
Debug	Debugging during development
Information	Normal application flow
Warning	Unexpected but handled issues
Error	Failures that stop an operation
Critical	Application crash or data loss

4. Practical Example – Logging in a Controller

Step 1: Inject ILogger<T>

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.Extensions.Logging;
```

```
[ApiController]
```

```
[Route("api/products")]
```

```
public class ProductsController : ControllerBase
```

```
{
```

```
    private readonly ILogger<ProductsController> _logger;
```

```
    public ProductsController(ILogger<ProductsController> logger)
```

```
    {
```

```
        _logger = logger;
```

```
    }
```

```
    [HttpGet("{id}")]
```

```
    public IActionResult GetProduct(int id)
```

```

{
    _logger.LogInformation("GetProduct started. ProductId: {ProductId}", id);

    if (id <= 0)
    {
        _logger.LogWarning("Invalid ProductId received: {ProductId}", id);
        return BadRequest("Invalid product id");
    }

    try
    {
        // Simulate data access
        var product = new { Id = id, Name = "Laptop", Price = 50000 };

        _logger.LogInformation("Product fetched successfully. ProductId: {ProductId}", id);
        return Ok(product);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error occurred while fetching product. ProductId: {ProductId}", id);
        return StatusCode(500, "Internal server error");
    }
}

```

Key Takeaways

- ILogger<T> is injected via DI

- Structured logging uses placeholders {ProductId}
 - Exceptions should always be logged with the exception object
-

5. Logging Configuration (appsettings.json)

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

Explanation:

- Logs **Information and above** for application code
- Logs **Warning and above** for framework logs

You can control logging levels via appsettings.json:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

- This config controls the minimum log level for different parts (namespaces) of your application.
 - ASP.NET Core uses a hierarchical logging system, so you can filter logs per namespace.
-

```

json
{
  "logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}

```

- |
- **"Default": "Information"**
 - This sets the default minimum log level for your whole application.
 - All logs with level Information, Warning, Error, Critical will be shown.
 - Debug and Trace messages will be ignored unless overridden.
-  **"Microsoft.AspNetCore": "Warning"**
 - This overrides the log level only for logs coming from Microsoft.AspNetCore (framework logs).
 - It suppresses noisy logs like info/debug from ASP.NET Core internals (e.g., middleware, routing).
 - Only Warning, Error, and Critical logs from this namespace will be shown.

6. Practical Example – Logging in Middleware

```

public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestLoggingMiddleware> _logger;

    public RequestLoggingMiddleware(
        RequestDelegate next,
        ILogger<RequestLoggingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task Invoke(HttpContext context)
    {
        _logger.LogInformation(

```

```

        "Incoming request: {Method} {Path}",
        context.Request.Method,
        context.Request.Path);

await _next(context);

_logger.LogInformation(
    "Response status code: {StatusCode}",
    context.Response.StatusCode);
}
}

```

7. Why Logging Is Preferred Over Console.WriteLine

Console.WriteLine	ILogger
No log levels	Supports log levels
Not structured	Structured logging
Hard to filter	Easy filtering
Not production-ready	Production-grade

8. Best Practices

- Use **ILogger<T>**, never static logging(Console)
- Do not log sensitive data (passwords, tokens)
- Use **structured logging**, not string concatenation
- Log exceptions with stack trace
- Configure log levels per environment

9. Summary

- Logging records application events for diagnostics and monitoring
- ASP.NET Core has built-in logging with extensible providers
- `ILogger<T>` is the primary interface
- Use appropriate log levels and structured messages

Third-party Loggers

- For advanced scenarios like writing logs to a file, database, or Elasticsearch, use
- Serilog
- Nlog
- log4net